

A one-page map of the whole project and every component. For depth: the [white paper](#) (math + sources), the [deck](#) (plain-English), the [technical docs](#) (how to run it), and the living [model ledger](#).

ABRA is a decision-support model family for **Pokémon Champions VGC, Reg M-B, best-of-one closed-sheet ladder**. It stores every public ladder replay and builds small, CPU-trainable models on that growing store. It runs in a browser with no build step.

You cannot reliably predict who wins from the two team sheets — even a player-rating model ties a coin. So ABRA does not sell outcome prediction. It supports *decisions* and grades every model with a proper score, a confidence interval, and an honest baseline. Wins are reported as wins; two honest negatives are reported as negatives.

Four experiments added knowledge to the model. All four measured a null. Two experiments changed what the model is optimising for. Both were large wins.

change	kind	result
take the best move instead of sampling it	objective	+12 points, 79.7% of decisive pairs
self-play policy improvement over the clone	objective	55.9%
four separate feature additions	knowledge	four nulls

RECONCILED 2026-07-31. *That 55.9% was measured on the **53-feature vector with switching OFF**. Repeating the experiment on the **56-feature vector with switching ON** gives **48.1%** [46.5, 49.8] over 9,728 paired games — a interval entirely below 50, i.e. self-play training made the policy worse. Both numbers stand as measurements of different configurations; neither generalises to 'self-play helps'. The difference is not explained, and three candidate causes are untested: switching exploration being harmful (consistent with the older 10-point switching loss), 36.5% drift over 18 iterations, or self-play eroding imitation-fitted features that were already good.*

The nulls survived the obvious check: an overdispersion test across teams reads ~1.00, against 1.169 for a known real effect, so they are genuine rather than a real effect hidden by team variety.

The constraint is the objective, not the knowledge. This is why the next item is retraining a model that already exists (DODUO, the pair-scoring layer, which lost at 42.0% fitted to *resemble humans* and has never been fitted to *win*), rather than adding more features to MAG.

A second, blunter lesson from the same day. Every integrity bug found had one shape: a fact reached one consumer and not the next. Priority blocking was in the artifact but not the simulator, so Sucker Punch beat a Farigiraf in every game ever simulated. A switch-in's own ability never reached the code that chooses the switch — measured over 40,001 matchups, declaring Intimidate, Drizzle or Drought changed nothing at all. And every mega forme carried no ability, no moves and no item, so **26% of the format scored as threatening nothing**. None of these were modelling disagreements. They were plumbing.

A third lesson, 2026-08-04, and it is about the rulers rather than the models. A result that does not record its own configuration cannot be checked by anyone, including the person who produced it. The R1 gate published "+2.91 [1.79, 4.04]"; recomputed from the only committed evidence it is **+0.456 [-0.717, +1.630] — UNDECIDED**. Nothing was falsified: the row dump recorded the answers and not the settings, so a run at exploration 0 and a run at exploration 1 left byte-compatible files that differ by nearly four accuracy points.

Auditing the sibling gates against the same standard found two more.

gate	published	what the evidence supports
R1 leaf accuracy	+2.91 [1.79, 4.04], PASSED	+0.456 [-0.717, +1.630], UNDECIDED
R2 leaf cost	5.83 ms median	reproduces only as arithmetic on itself, and it timed <code>explore=0</code> at a 20-turn horizon while the shipped leaf runs <code>explore=1.0</code> at 60
R3 divergence	72.9% over 70 decisions	recomputes exactly — from two fields in the same file. Its control was printed and never stored , and the gate's own verdict branches on that control

Every gate now writes a sidecar (`engine/run_stamp.js`) recording budget, exploration rate, horizon, content digests of every source it reads, the commit, and whether the tree was dirty. Older artifacts carry one reconstructed from the commit that contained them, labelled inferred rather than observed.

Model	What it is	Status	Headline result
MEDICHAM	Hand-written doubles doubles-battle simulator	⚠️ Being replaced	Within 5% of the Smogon calculator on 31 scenarios, but disagrees with the OFFICIAL Champions engine by 31.1 points of win probability. ADR-001: becomes a lookup over precomputed tables. Mechanics census 181 live of 186 probed, 5 missing with reasons (<code>data/mechanics-census.json</code> , wires 82–89 landed 3.40.0); generated interaction matrix 98.7% — 999 of 1,012 live carrier x reactor cases, multi-turn field axis 156/156 (<code>data/interaction-matrix.json</code>); damage differential 1/150 , the one row a documented harness-layer artifact (Disguise); two-rulebook collision ratchet 2 clashes / 151 comparable facts (<code>data/rulebook-collision.json</code>)
GURU	Meta matchup matrix from real outcomes	⚠️ No decisive cells that survive multiplicity	<code>data/guru-matchups.json</code> , 2026-07-31, 5,265 clean games / 12 archetypes / 144 cells. 6 directed = 3 distinct matchups clear a 95% test one at a time, and ZERO survive FDR at q=0.05 or Bonferroni — 66 pairs, 3.3 expected by chance, 3 observed, smallest exact p 6.1e-3 against a BH threshold of 7.6e-4. Predictive test 0.7124 vs a coin 0.6931 over 1,053 held-out games — worse than a coin . Descriptive structure only. (This row read <i>1,124 clean games, 11 archetypes, 0.735</i> until 2026-08-04, from a superseded run; the verdict is unchanged.)
XATU	Opponent set + next-move belief	✅ Built	Top-1 36% / top-3 72% on held-out human moves (beats its baselines)
PORY	Mid-game win-probability value net	⚠️ Contribution unclear	Log-loss 0.6298 95% CI [0.6125, 0.6456] vs coin 0.6931 and vs the material heuristic 0.655 (regenerated 2026-07-31; the previously published 0.567 predated the current quality filter) — but its features ARE the material state, and it loses to a two-feature baseline (alive_diff+hp_diff 0.5822 vs PORY 0.5840, same estimator). Report the gain over MATERIAL, not over a coin. See <code>engine/pory_baseline.py</code>
CHOMP	Bring-4 / lead-2 team-preview engine	✅ Ships (standalone)	Exact-damage picker; CHOMP-EV proof: brings tie a coin (honest null)
SLOWKING	Team-preview Nash (mixed strategy)	✅ Built	Equilibrium << exploitable than uniform; playstyle cycle is suggestive on small samples
KADABRA	Replay coach	✅ Works offline	Per-turn "you're at X%" from PORY
DITTO	Team optimiser	⚠️ Pivoting	Objective de-biased to validated damage (was optimising a backwards signal)
ALAKAZAM	In-battle decision engine (capstone)	➡️ In development	Belief + search + learned value; built last on the inputs above
MEW	Self-play data engine	✅ Built	Runs the OFFICIAL Champions engine against itself on real observed teams. 1,000 games, 13/13 validation checks, mirror 51.0% CI [45.4,

Model	What it is	Status	Headline result
			56.6]
MAGNEMITE (MAG)	The in-battle policy that reads the board	Built, and improving by self-play (3.28.0)	Conditional logit over 53 features , fitted to 146,910 real human clicks from 6,091 clean open-sheet games. Held out by game: top-1 33.6% against the behaviour clone's 27.1%. It now DOES decide switches and DOES run a real damage calculation — both were listed here as missing and both became false. Still one ply, still no model of the opponent's move
WOBBUFFET	Exploitability of MAG — hill-climb a counter over MAG's own weights	✗ NOT MEASURED	There is no exploitability number for this project (2026-08-04). The published $\sim 63.2\%$ [56.6, 69.3], mirror 47.5% $\sim$ is retracted : 17 features against the 58 we ship, an engine 25 wire-fixes old, computed before the quality filter existed. The 58-feature re-run is void — <code>data/policy-weights.json</code> was refitted at 22:15:24 UTC while it was running and <code>engine/medicham2-browser.js</code> changed content twice more afterwards. Separately its hill-climb accepted 1 of 24 steps and would have been uninformative anyway. <code>engine/exploit.js</code> stamps nothing about what it read, which is why none of this was visible to it. See <code>docs/SEARCH.md</code> §R8
DUSK	Endgame exact solver	 Roadmap	Solves small boards ($\leq 2v2$, 1v1) perfectly — sharpens ALAKAZAM's endgame and gives clean training targets for PORY
HYPNO	Opponent read / exploitability dial	 Roadmap	Estimates opponent strength + predictability; tells ALAKAZAM when to play safe (vs strong) or exploit (vs weak/predictable)
ROLES	Multi-label team composition (26 roles)	 Built	Role-pair matrix pools data to median cell n=20 across 1,051 cells (vs old single-label $n=11-18$) — the 7,971 once published was retracted in 2.7.0; preview roles tie a coin (honest null)
WAR	Wins Above Replacement (species RAPM)	 Null	Withdrawn 2026-07-25. Beat a coin only on the unfiltered store (0.6860). On clean games: 0.7048 vs coin 0.6931, accuracy 0.502 — the signal was four bots playing one team 1,446 times
NMF	Emergent roles / archetypes	 Rank not defensible	Rank 6 ships, but the project's own criterion (<code>engine/nmf_rank.py</code> , bootstrap factor stability, cf. Brunet et al. 2004) selects rank 4 — and rank 6 scores -0.107 excess over null , i.e. its factors are <i>less</i> reproducible across resamples than factors fitted to shuffled data. The old justification here was reconstruction error 0.53, which that same script states cannot select a rank (it falls monotonically by construction). A team is a <i>blend</i> , learned not hand-labelled — but the number of blends is not currently defended

Multiplicity, corrected 2026-07-31. The fit reports a 95% interval for all 56 features, so at alpha 0.05 about **2.8 of them clear zero by chance alone**. The family is **every feature in the shipped fit**, because every one is reported to the reader — choosing a smaller family after seeing which are large is the practice the correction exists to prevent. Uncorrected, **53** clear zero. Under **Benjamini–Hochberg** (FDR, 1995) **53** survive; under **Bonferroni** (FWER) **49**. Nothing significant uncorrected fails the FDR correction, so the headline count is not an artefact of having looked at 56. Computed by `engine/weight_multiplicity.js` → `data/weight-multiplicity.json`. **This says which weights are distinguishable from zero. It says nothing about whether an imitation-fitted weight is evidence about WINNING** — a separate and larger question this project has measured going the other way.

A phrasing the filter itself mandates. `require_full_bring` conditions on game length: measured 2026-07-31, the games it keeps are **1.71x longer** on average (7.4 vs 4.3 mean turns; 19,589 kept vs 8,713 dropped). Every bring statistic in this project is therefore *"the bring, among games long enough to show it"*, which is not the same as "the bring". `data/quality-filter.json` states this at the point of filtering and requires it to be said downstream; this is that.

The **store** (every real game) feeds **GURU** (meta), **XATU** (belief), **PORY** (value), and **MEDICHAM** (damage). **SLOWKING** solves the preview; **CHOMP** picks the bring; **KADABRA** coaches a replay with PORY. **ALAKAZAM** is the capstone that assembles belief + search + value into the win-%-optimal move, built last. Every change updates the code, this summary, the white paper, the deck, the technical docs, and the CHANGELOG in the same pass.

Piece	Repo	Live
ABRA (models + site)	github.com/willhoop/abra	willhoop.github.io/abra/app/
CHOMP (bring engine)	github.com/willhoop/chomp	Showdown userscript
Portfolio	github.com/willhoop/willhoop.github.io	willhoop.github.io

Collected (closed-sheet Bo1 ladder)	see data/live.js — generated on every refresh, growing hourly (hardcoded sizes retracted, S13)
Usable after the quality filter	1,124 (12.8%)
Self-play (MEW, official engine)	1,000 — separate file, never pooled
Open-team-sheet archive	4,167 (MIT, 2026-06-17..20) — separate file, different information regime
Smogon official priors	283 species, whole-ladder aggregate

`meta-usage.json` publishes both, because they answer different questions:

competitive	garchomp , incineroar , kingambit , sinistcha , whimsicott , basculegion
ladder	garchomp , whimsicott , kingambit , basculegion , charizard , incineroar

Competitive is what humans choose when trying — right for tournament prep and for any claim about the game. **Ladder** is what you actually face: three in four STORED games involve a bot. That is a property of what gets uploaded rather than of the ladder: bot-team species are over-represented in the scrape by a mean of +8.3 points against Smogon whole-ladder statistics, while other top species run -2.9. The true share of bot opponents is lower than the store implies, but it is not small. Charizard sits at 25.7% on the ladder view and outside the competitive top six because it is on the bot team. Consumers must say which they used.

Predicting the match winner from sheets is a coin flip in this format — and the previously published 55.0% skill ceiling was itself measured with bots included. Removing them gives 52.4%, an interval that contains a coin flip. Every preview-level model now sits at that ceiling: JOLTEON, roles, CHOMP-EV, and as of 3.2.0 **WAR, whose result is withdrawn.**

Most results here are also **underpowered**: 1,124 clean games can only detect an edge of ~4.2 accuracy points, and a 2-point effect needs ~4,900. `engine/eval_harness.py` now refuses to report a null without stating what it could have seen.

The one load-bearing win is the **validated damage engine** — 31/31 within 2% against two independent oracles. PORY was the other, until 2026-07-25 showed it loses to a two-feature material baseline. The project's two genuine contributions are the ones it treats as plumbing: **behavioural bot detection**, and the **measurement discipline** that dissolved WAR, the 55% ceiling and GURU's matchup matrix in a single day.

Measured 2026-07-25 against Smogon's whole-ladder statistics for the same format and month (1,163,315 battles vs the count generated into `data/live.js`):

	mean difference, uploaded vs whole ladder
The five bot-team species	+8.3 points
Every other top species	-2.9 points

75% of *stored* games involve a detected bot. That is a fact about **what gets uploaded**, not about the ladder — bots save replays far more readily than humans do. Any statement of the form "three in four of your opponents are bots" is therefore an overestimate and should not be made from this store.

This also bounds the earlier upload-bias result. Comparing our open-team-sheet Bo3 games against the whole Bo3 ladder gave a mean absolute difference of only 1.84 points — but that corpus contains almost no bots (29 named-bot sides in 4,167 games). So **human** upload bias is small; **bot** upload bias is large, and the closed-sheet Bo1 store carries the latter.

item	state
engine release	5fc1f711a0e3 , 12 files frozen, Showdown 20ad99ff . A measurement reads the snapshot, not the live tree, so the divisions can run concurrently.
provenance method	content digests , no longer mtime. 0 artifacts verified by content, 92 by mtime alone — printed every run, ratcheted downward by a named list.
exploitability	no figure . 63.2% retracted on its own merits; the 2026-08-04 re-run is void: true .
mirror control	49.7% [46.2, 53.2] , n=782 — survives the void run and retires the seat-asymmetry worry.
MAG refit	ran; moved nothing measurable . Weather fix +0.048 top-1 [0.009, 0.093]; the refit itself −0.074 [−0.155, +0.004] against a 0.192-point noise floor.
open, needs a decision	the fit sees {nature, item} ; the player sees {nature, item, ability, moves} . 50.47% of trained decisions , 99.75% of games.